

Jobflow Issues & Lessons Learned

The real debugging journey behind the polished application

Documentation package
Prepared from the Jobflow V3 codebase and current product direction.

Jobflow is a Django-based job application tracking workspace designed to turn scattered applications into a clear, actionable pipeline.

Contents

Section	What it covers
Why this log exists	Why debugging history matters
Major issues	Six problems and fixes
Debugging method	A repeatable troubleshooting process
Compatibility	What the Django/MySQL issue taught
Product impact	How issues improved the product
Interview talking point	How to explain the journey
Future risks	What production adds
Final lesson	The engineering mindset

1. Why keep an issues log?

A strong portfolio project is not only a polished final screen. Debugging history shows how the developer reacts when reality disagrees with the plan. This log records the main issues encountered while evolving Jobflow into the V3 portfolio-ready version.

2. Major issues

01. MySQL access denied

Symptom: "Access denied for user 'root'@'localhost' (using password: NO)."

Cause: The database configuration was not providing the expected password.

Fix: Corrected the local database configuration/credentials.

Lesson learned: Separate database configuration failures from application-code failures.

02. Frontend felt too simple

Symptom: The functionality worked, but the product did not yet feel like a modern SaaS application.

Cause: Visual hierarchy, cards, motion, spacing, and identity were underdeveloped.

Fix: Reworked the UI into the Jobflow design system.

Lesson learned: Functional correctness and product polish are different deliverables.

03. Typography was too small

Symptom: The redesigned interface was visually polished but still felt hard to read.

Cause: Density had been prioritized too strongly.

Fix: Increased headings, body text, navigation, card content, and spacing.

Lesson learned: Readability is a product requirement.

04. Different browser windows showed different designs

Symptom: One Chrome window appeared to show an older build.

Cause: Different URLs/processes or cached assets can create this symptom.

Fix: Verified the running server/URL and refreshed the correct build.

Lesson learned: Check the exact running version before debugging code.

05. PowerShell blocked venv activation

Symptom: The activation script was blocked by Windows execution policy.

Cause: PowerShell prevented the script from running.

Fix: Used the venv's Python executable directly instead of changing system policy.

Lesson learned: Environment problems often have safe, reversible workarounds.

06. Django/MySQL compatibility mismatch

Symptom: A fresh environment reported that MySQL 8.4+ was required while MySQL 8.0.46 was installed.

Cause: A newer Django version had been installed than the previously working environment.

Fix: Preserved the known-compatible environment strategy rather than immediately upgrading MySQL.

Lesson learned: Pin dependencies and upgrade infrastructure deliberately.

3. Debugging method

- Start with the exact error message.
- Identify the layer: browser, frontend, Django, database, environment, or deployment.
- Keep a working baseline before large changes.
- Prefer reversible configuration changes before infrastructure upgrades.
- Use Django's system check as a fast validation step.
- After routing changes, test public and authenticated routes separately.

4. Compatibility lesson

The MySQL 8.0.46 versus newer Django requirement showed why projects should not rely on unpinned “latest” packages. Reproducibility matters: identify the working versions, pin them, then upgrade intentionally with tests.

5. How problems improved the product

Problem	Resulting improvement
Visual simplicity	Modern Jobflow design system and richer states
Small typography	Readable hierarchy and larger cards/navigation
Portfolio privacy	Public read-only Demo Mode with fictional data
Environment mismatch	More deliberate dependency compatibility
Unclear public entry point	Dedicated landing page + /app/ private area

6. Interview talking point

“The hardest part was not writing CRUD. It was making the application reliable across environments while improving the product experience. I learned to distinguish database configuration errors, Windows environment issues, package compatibility, browser version confusion, and UI design problems — and each issue led to a concrete improvement.”

7. Future production risks

- Production database migration and schema compatibility.
- Static-file collection and hosting configuration.
- Secure cookies, CSRF trusted origins, and HTTPS.
- Backups and recovery.
- Rate limiting and stronger account protections.

- Automated tests and CI.

8. Final lesson

The valuable outcome is not that every bug disappeared. It is that each bug became a repeatable engineering lesson: isolate the layer, reproduce the problem, change one thing at a time, verify, and document the result.